МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 1

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Основы OpenMP: параллельные области, распределение работ, синхронизация потоков»

Работу выполнил
Студент гр. 4404
Завалин М.Г.

Казань, 2026

Цель работы:

Изучение основ параллельного программирования для систем с общей памятью: параллельные области и идентификация потоков, распределение итераций цикла между потоками (типы расписания schedule), примитивы синхронизации потоков. В ходе работы выполнены задачи 1–3 методических указаний «Параллельное программирование на OpenMP». Программы написаны на языке C с использованием директив компилятора и библиотечных функций OpenMP (pragma omp, omp_get_thread_num, omp_set_num_threads и др.).

1. Задание

Задача 1. Написать программу, где каждый поток печатает свой идентификатор, количество потоков всего и строчку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Задача 2. Написать программу, в которой объявлен массив из 16000 элементов, инициализированный так, что значение элемента равно его порядковому номеру. Создать результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0. Запустить программу с 8 потоками при различных типах распределения работ.

Задача 3. Модифицировать задачу 1 так, чтобы потоки печатали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения; необходимо реализовать и продемонстрировать как можно больше из них.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком

решающие задачу; каждый запуск, о котором говорится в задаче (разное число потоков, разные типы распределения, разные версии программы), подтверждается скриншотом с видимой командой запуска и полным выводом.

2. Ход работы

2.1. Задача 1. Параллельная область и идентификация потоков

Программа запрашивает у пользователя число потоков N, затем с помощью директивы #pragma omp parallel num_threads(N) создаёт параллельную область из N потоков. Каждый поток получает свой идентификатор вызовом omp_get_thread_num() и общее число потоков вызовом omp_get_num_threads(), после чего печатает строку вида «Thread X of Y: Hello World».

Код программы:

```
#include <iostream>
#include <omp.h>
int main() {
    const int NUM_THREADS = 8;
    omp_set_num_threads(NUM_THREADS);
#pragma omp parallel
    {
        int tid = omp_get_thread_num();
        int nthreads = omp_get_num_threads();
#pragma omp critical
        std::cout << "Thread " << tid << " of " << nthreads <<
std::endl;
    }
    return 0;
}
```

Ответ на вопрос задачи: вывод не всегда идентичен. Потоки выполняются параллельно, и порядок, в котором они достигают оператора printf, определяется планировщиком операционной системы, поэтому от запуска к запуску порядок строк может меняться. Множество строк (по одной от каждого потока) одинаково во всех запусках, но их последовательность программой не задаётся, так как синхронизация, фиксирующая порядок вывода, отсутствует.

Для подтверждения выполнено три запуска программы с 8 потоками. На рисунке 1 показан первый запуск программы с 8 потоками. Видна команда запуска и полный вывод: каждый из 8 потоков напечатал свой идентификатор, общее число потоков и строку «Hello World».

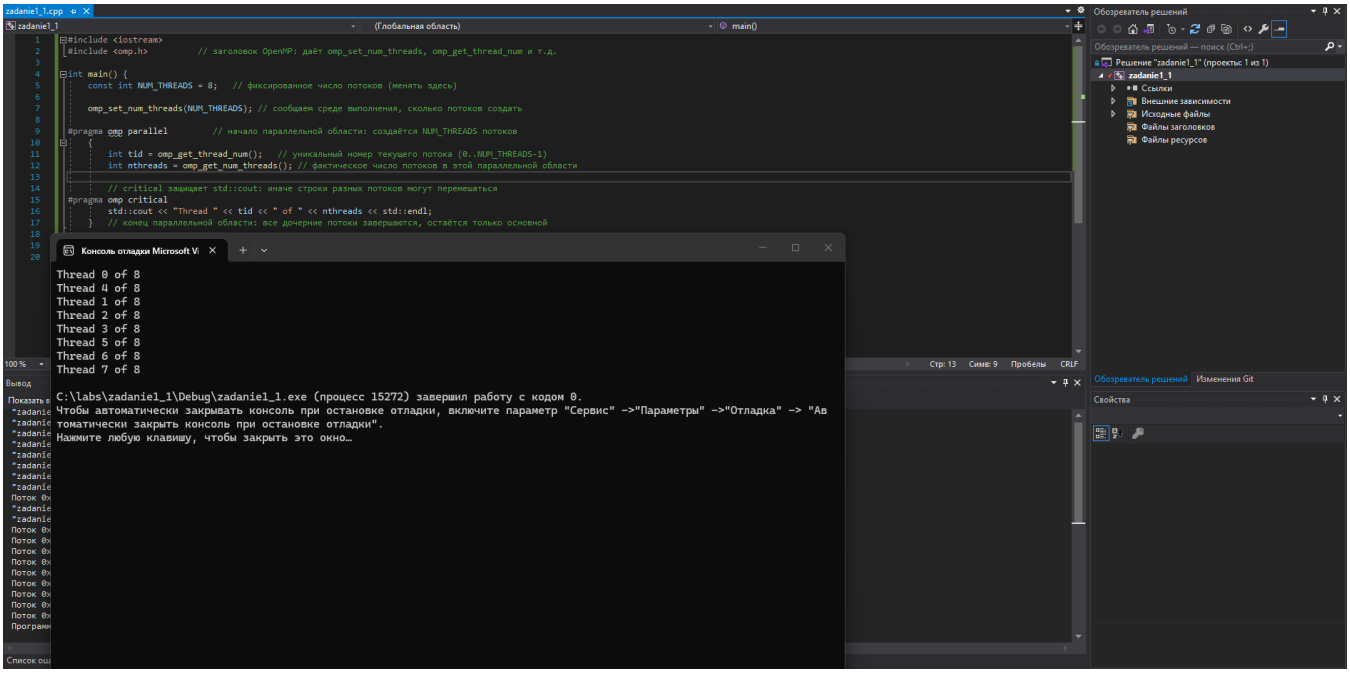


Рисунок 1 — Задача 1: запуск с 8 потоками, запуск № 1

На рисунке 2 показан второй запуск той же команды. Порядок строк может отличаться от рисунка 1, так как порядок выполнения потоков не определён. Если порядок совпал, это не гарантирует детерминированность вывода — программа порядок не фиксирует.

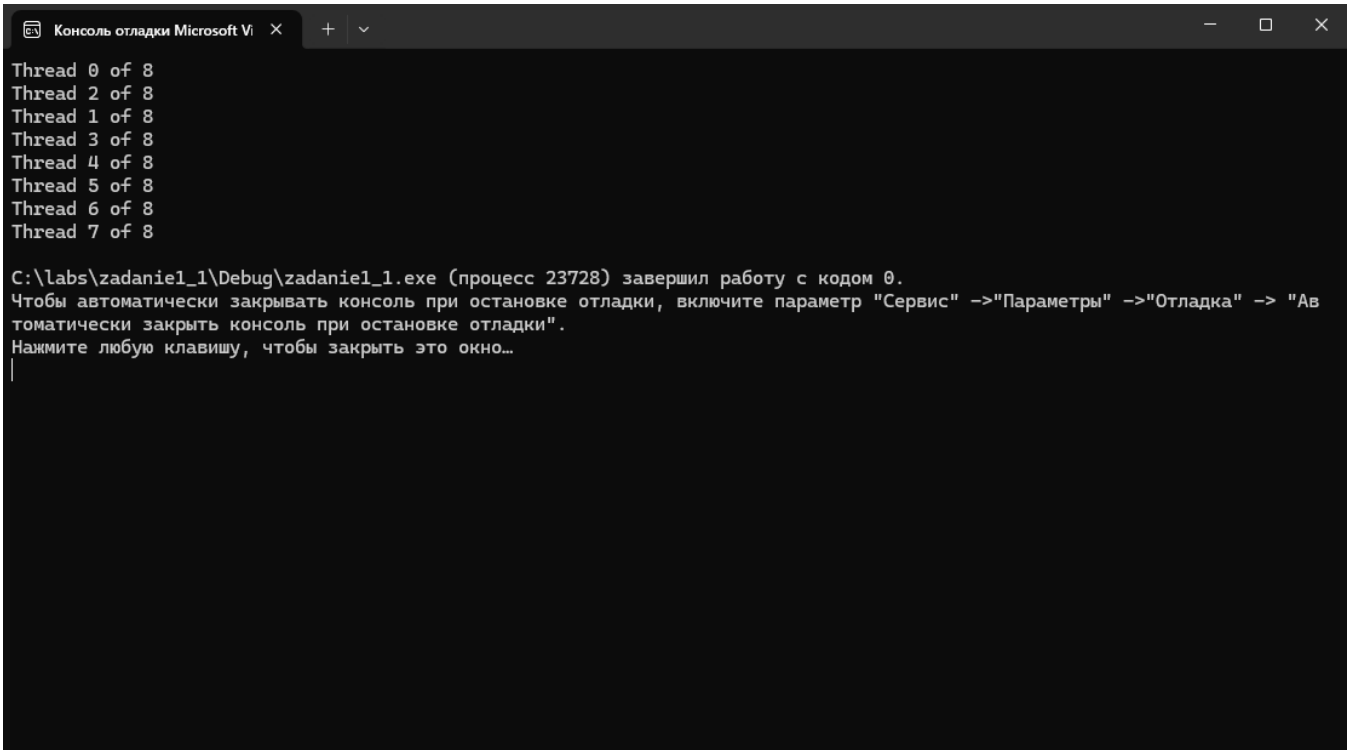


Рисунок 2 — Задача 1: запуск с 8 потоками, запуск № 2

На рисунке 3 показан третий запуск. Сравнение рисунков 1–3 показывает, что набор строк вывода во всех запусках одинаков, а их порядок может изменяться, что подтверждает ответ на вопрос задачи.

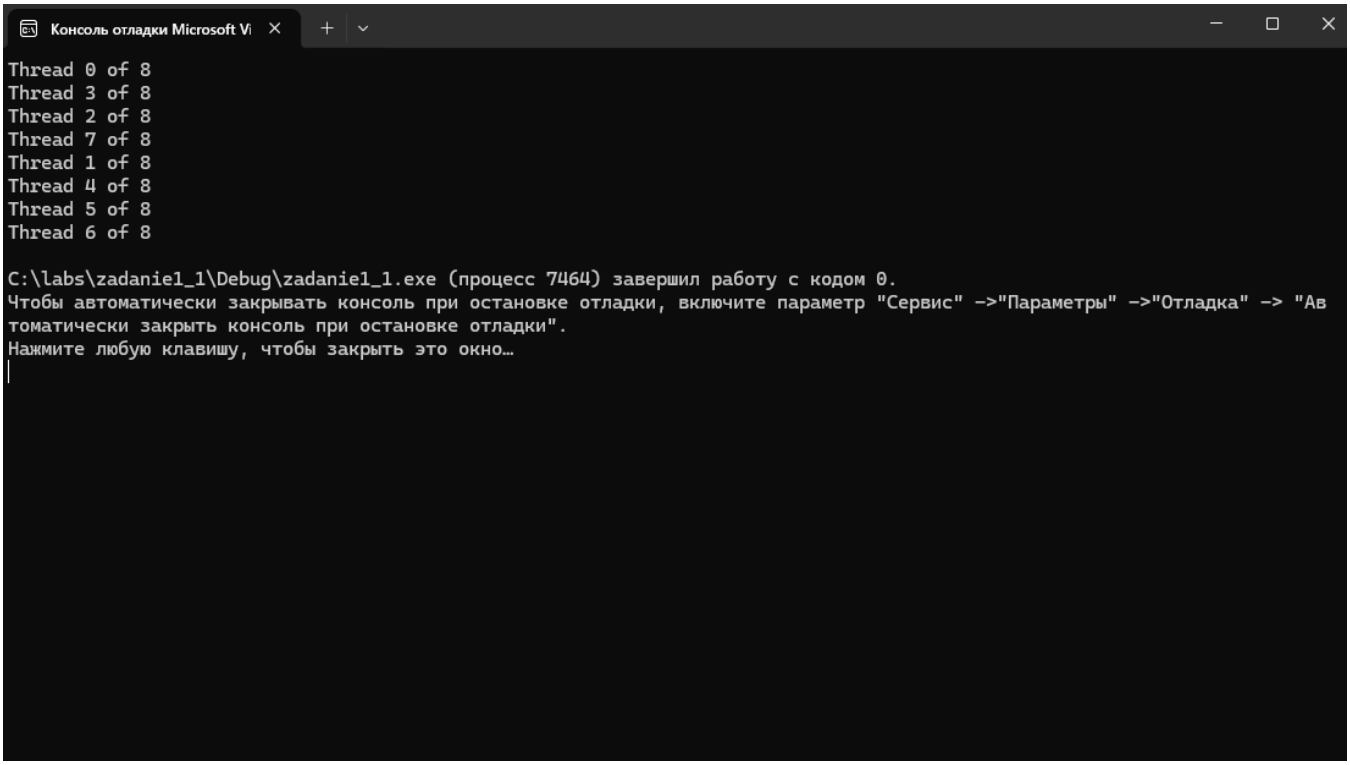


Рисунок 3 — Задача 1: запуск с 8 потоками, запуск № 3

2.2. Задача 2. Обработка массива при различных типах распределения работ.

Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: b[i] = (a[i-1] + a[i] + a[i+1])/3.0. Запустите программу с 8-ю процессами при различных типах распределения работ.

Код программы:

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <cmath>
#include <omp.h>
int main() {
   const int    N = 16000; // размер массива
   const int    NUM_THREADS = 8;
   const double EPS = 1e-6;
   omp_set_num_threads(NUM_THREADS);
   std::vector<double> a(N), b(N);
   for (int i = 0; i < N; ++i) {
      a[i] = static_cast<double>(i);
   }
   b[0] = a[0];
   b[N - 1] = a[N - 1];
   const char* schedules[] = { "static", "dynamic", "guided" };
   const int   num_sched = 3;
   for (int s = 0; s < num_sched; ++s) {
      double t_start = omp_get_wtime();
      if (s == 0) {
#pragma omp parallel for schedule(static)
         for (int i = 1; i < N - 1; ++i) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
         }
      }
      else if (s == 1) {
#pragma omp parallel for schedule(dynamic)
         for (int i = 1; i < N - 1; ++i) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
         }
      }
      else {
#pragma omp parallel for schedule(guided)
         for (int i = 1; i < N - 1; ++i) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
         }
      }
      double t_end = omp_get_wtime();
      double expected1 = (a[0] + a[1] + a[2]) / 3.0;
```

```
            double expectedN = (a[N - 3] + a[N - 2] + a[N - 1]) / 3.0;
            bool ok = (std::abs(b[1] - expected1) < EPS) &&
                (std::abs(b[N - 2] - expectedN) < EPS);
            std::cout << std::fixed << std::setprecision(6);
            std::cout << "Schedule = " << schedules[s]
                << ", threads = " << NUM_THREADS
                << ", time = " << (t_end - t_start) << " sec"
                << ", check = " << (ok ? "OK" : "FAIL") << "\n";
            std::cout << "  b[1]    = " << b[1] << " (expected " << expected1 << ")\n";
            std::cout << "  b[N-2]  = " << b[N - 2] << " (expected " << expectedN << ")\n";
        }
        return 0;
    }
```

Внутренний цикл обрабатывает **15998 итераций** (i = 1 … N−2). Формула среднего применима только к внутренним элементам: для i = 0 потребовался бы a[−1], для i = N−1 — a[N], которых не существует. Поэтому крайние элементы b[0] и b[N−1] копируются из a до цикла и остаются неизменными.

Программа последовательно запускает цикл с тремя разными типами расписания: schedule(static), schedule(dynamic) и schedule(guided). Число потоков фиксировано (NUM_THREADS = 8) и устанавливается через omp_set_num_threads() до входа в параллельные области. Для каждого запуска замеряется время выполнения через omp_get_wtime().

После каждого прогона выполняется проверка корректности. Сравниваются два «эталонных» значения — первое и последнее из тех, к которым применима формула:

- b[1] — первый элемент, посчитанный по формуле;
- b[N−2] — последний элемент, к которому формула применима.

Именно b[N−2], а не b[N−1], берётся для проверки: b[N−1] — это копия a[N−1], а не среднее, и проверять на нём формулу бессмысленно. Модуль разности сравнивается с допуском EPS = 1e-6, результат выводится как OK или FAIL.

Программа обрабатывает 15998 внутренних итераций и использует #pragma omp parallel for с клаузой schedule(...) для распределения этих итераций между потоками. В коде применены три типа расписания.

schedule(static)**.** Итерации заранее, ещё на этапе компиляции, делятся между потоками непрерывными, почти равными блоками. При 8 потоках и 15998 итерациях каждый поток получает около 2000 итераций подряд: например, поток 0 обрабатывает индексы 1…2000, поток 1 — 2001…4000 и так далее. Распределение фиксируется до начала цикла, поэтому накладные расходы на планирование минимальны. Недостаток подхода проявляется, если итерации имеют разную трудоёмкость: часть потоков завершит работу раньше и будет простаивать. В данной задаче все итерации одинаковы по стоимости, поэтому static работает эффективно.

schedule(dynamic)**.** Итерации разбиваются на блоки (по умолчанию размер блока равен 1). Поток берёт очередной блок из общей очереди, обрабатывает его и запрашивает следующий. Если один поток работает медленнее, другие забирают оставшиеся блоки — балансировка загрузки происходит автоматически. Поскольку

очередь общая, доступ к ней защищён механизмом синхронизации (аналог критической секции внутри реализации OpenMP), что даёт дополнительные накладные расходы по сравнению со static. Чем меньше размер блока, тем точнее балансировка, но тем выше overhead на выдачу блоков.

schedule(guided)**.** Как и dynamic, потоки берут блоки из общей очереди по мере освобождения. Отличие — в размере блока: первые блоки крупные (примерно «оставшаяся работа / число потоков»), а каждый следующий уменьшается по мере обработки, но не опускается ниже минимального значения. Благодаря этому в начале работы накладные расходы на синхронизацию минимальны (крупные блоки), а к концу достигается хорошая балансировка (мелкие блоки дораздают остатки). guided представляет собой компромисс между static (нет расходов на очередь, но нет балансировки) и dynamic (хорошая балансировка, но много синхронизации).

Работа программы представлена на рисунке 4.

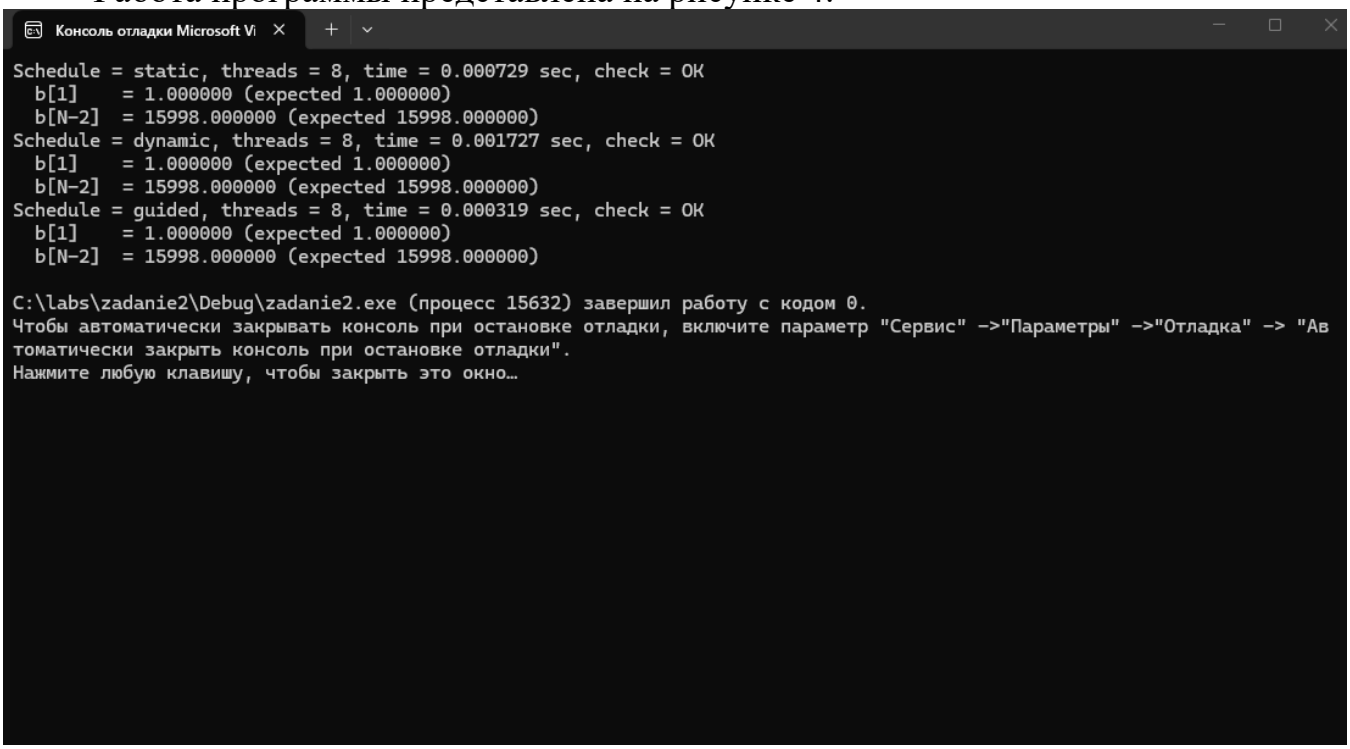


Рисунок 4 — Задача 2: вывод программы

2.3. Во всех трёх случаях результат совпадает: b[1] и b[N−2] соответствуют ожидаемым значениям с точностью до EPS, проверка выводит OK. Итерации цикла независимы: каждая итерация i читает элементы a[i−1], a[i], a[i+1] из неизменяемого массива a и записывает результат только в свой элемент b[i]. Гонок данных не возникает, а значит, итоговый массив b полностью определяется формулой и не зависит от того, в каком порядке потоки обрабатывают итерации. Тип расписания влияет лишь на то, какие именно итерации достаются каждому потоку и сколько времени занимает выполнение, но не на результат. Все три типа распределения дают корректный результат. На массиве из 16000 элементов с одинаковой трудоёмкостью итераций заметного различия во времени между static, dynamic и guided не наблюдается. Для наглядного сравнения типов

распределения требуется задача с неравномерной нагрузкой на итерации — тогда dynamic и guided покажут преимущество в балансировке перед static

2.4. Задача 3. Вывод идентификаторов потоков в обратном порядке

Для фиксации порядка вывода необходимы средства синхронизации. В задаче требуется модифицировать программу из задачи 1 так, чтобы потоки печатали свои идентификаторы в обратном порядке: от N-1 до 0.

1 способ - Каждый поток записывает свой идентификатор в разделяемый массив ids[tid] = tid. После этого в параллельной области запускается цикл for (i = 0; i < n; ++i) с директивой #pragma omp for ordered, внутри которого стоит #pragma omp ordered. Итерации ordered-цикла выполняются в порядке возрастания i, но тело печатает ids[n − 1 − i] — то есть массив обходится с конца. В результате строки выводятся в порядке 7, 6, 5, …, 0.

Механизм синхронизации. Директива #pragma omp ordered. Она гарантирует, что итерации ordered-цикла выполняются строго в порядке их номеров, и что внутри ordered-секции одновременно находится только один поток.

```
#include <iostream>

#include <omp.h>
#include "ways.h"
void way1(int n) {
   int ids[64] = { 0 };
#pragma omp parallel num_threads(n)
   {
      int tid = omp_get_thread_num();
      ids[tid] = tid;
#pragma omp for ordered
      for (int i = 0; i < n; ++i) {
#pragma omp ordered
         std::cout << "Thread " << ids[n - 1 - i]
            << " of " << n << "\n";
      }
   }
}
```

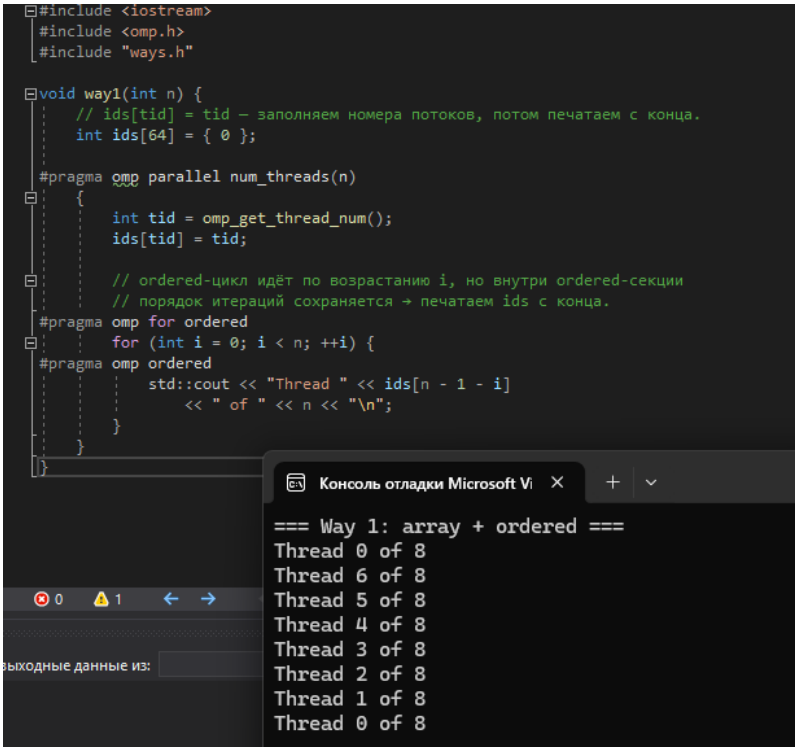


Рисунок 5 — Код способа 1: массив + ordered (way1.cpp)

Результат работы. Печать строго в порядке 7, 6, 5, …, 0.

2 способ - Вводится разделяемая переменная counter, инициализированная значением n. Каждый поток входит в критическую секцию, уменьшает counter на единицу и печатает полученное значение. Поскольку декремент и печать выполняются в одном критическом блоке, каждый поток получает уникальное число от n−1 до 0, и строки выводятся в обратном порядке.

Механизм синхронизации**.** Директива #pragma omp critical. Она гарантирует, что только один поток одновременно выполняет блок внутри неё, поэтому доступ к counter защищён от гонки данных. По сравнению с атомарной операцией критическая секция имеет более высокие накладные расходы — каждый вход и выход сопровождается захватом и освобождением блокировки (аналог мьютекса).

Работа программы представлена на рисунках 6.

```
#include <iostream>
#include <omp.h>
#include "ways.h"
void way2(int n) {
    int counter = n;
#pragma omp parallel num_threads(n)
    {
        {
            --counter;
            std::cout << "Thread " << counter
                << " of " << n << "\n";
        }
    }
}
```

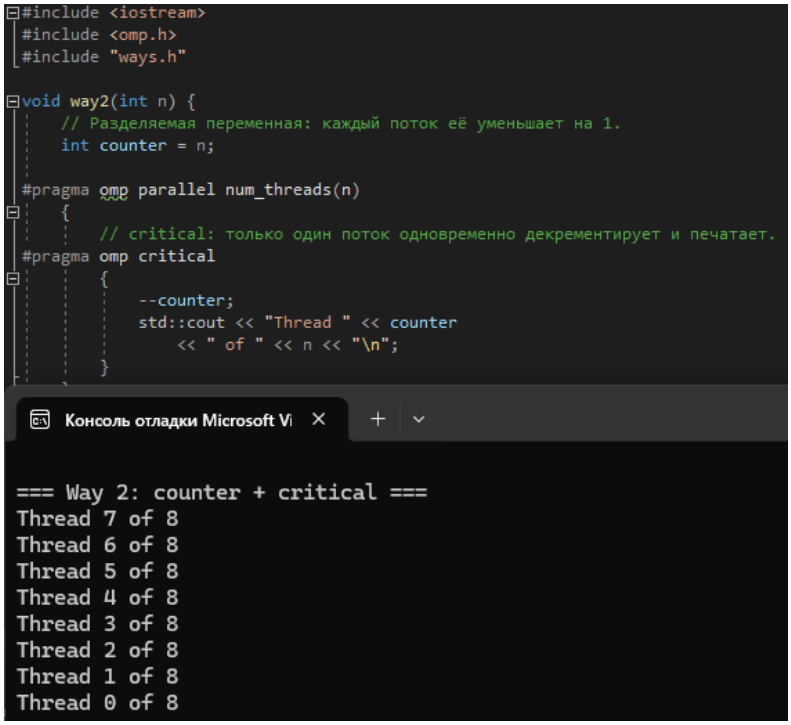


Рисунок 6 — Код способа 2: критическая секция (way2.cpp)

3 способ - Логика похожа на способ 1, но без промежуточного массива. Сразу в параллельной области запускается цикл for (i = n − 1; i >= 0; --i) с директивой #pragma omp for ordered, внутри которого стоит #pragma omp ordered. Директива ordered сохраняет порядок итераций, но сами итерации идут в обратную сторону — от n−1 до 0. Поэтому строки печатаются сразу в нужном порядке, без массива.

Механизм синхронизации. Та же директива #pragma omp ordered. Отличие от способа 1 — в направлении цикла: ordered-итерации выполняются в порядке номеров итераций, а мы заранее задали их в обратном порядке.

Работа программы представлена на рисунке 7.

```
#include <iostream>
#include <omp.h>
#include "ways.h"
void way3(int n) {
#pragma omp parallel num_threads(n)
    {
#pragma omp for ordered
        for (int i = n - 1; i >= 0; --i) {
#pragma omp ordered
            std::cout << "Thread " << i
                << " of " << n << "\n";
        }
    }
}
```

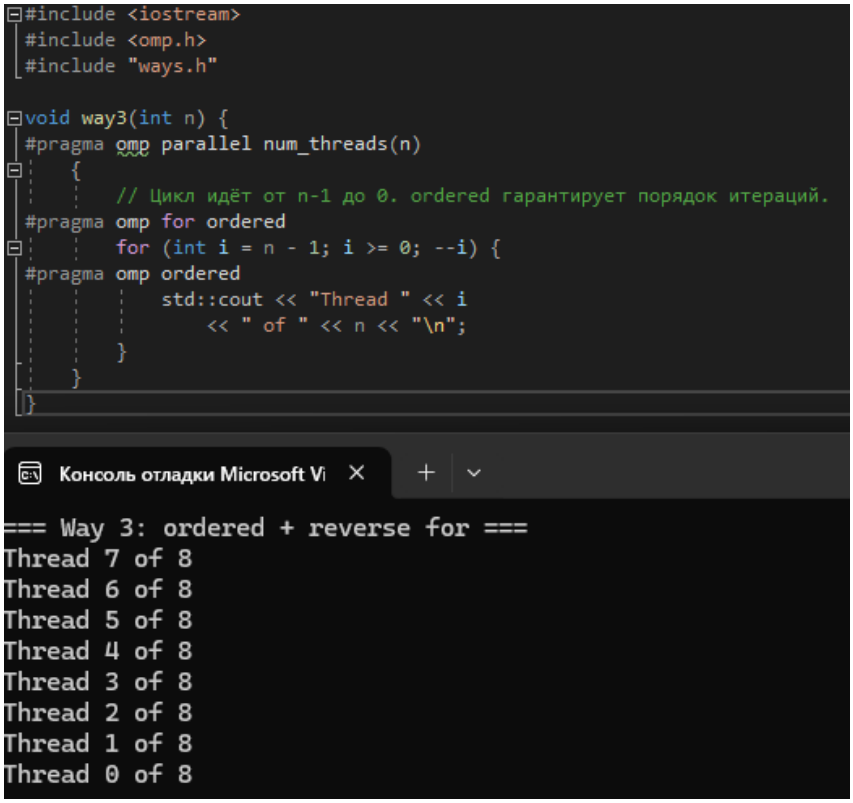


Рисунок 7 — Код способа 3: обратный ordered-цикл (way3.cpp)

4 способ – Потоки сначала собирают свои идентификаторы в разделяемый массив ids[tid] = tid, затем синхронизируются барьером, а после этого один поток (#pragma omp single) печатает массив в обратном порядке.

Механизм синхронизации. Два примитива:

- #pragma omp barrier — гарантирует, что все потоки завершили заполнение массива, прежде чем кто-либо начнёт печатать.
- #pragma omp single — блок выполняет ровно один (любой) поток; остальные его пропускают и ждут на неявном барьере в конце single.

Таким образом, к моменту печати массив уже полностью заполнен, а строки выводит единственный поток — порядок гарантирован.

Работа программы представлена на рисунке 8.

```
#include <iostream>
#include <omp.h>
#include "ways.h"
void way4(int n) {
   int ids[64] = { 0 };
#pragma omp parallel num_threads(n)
    {
       int tid = omp_get_thread_num();
       ids[tid] = tid;
#pragma omp barrier
#pragma omp single
        {
```

```
            for (int i = n - 1; i >= 0; --i)
                std::cout << "Thread " << ids[i]
                << " of " << n << "\n";
        }
    }
}
```

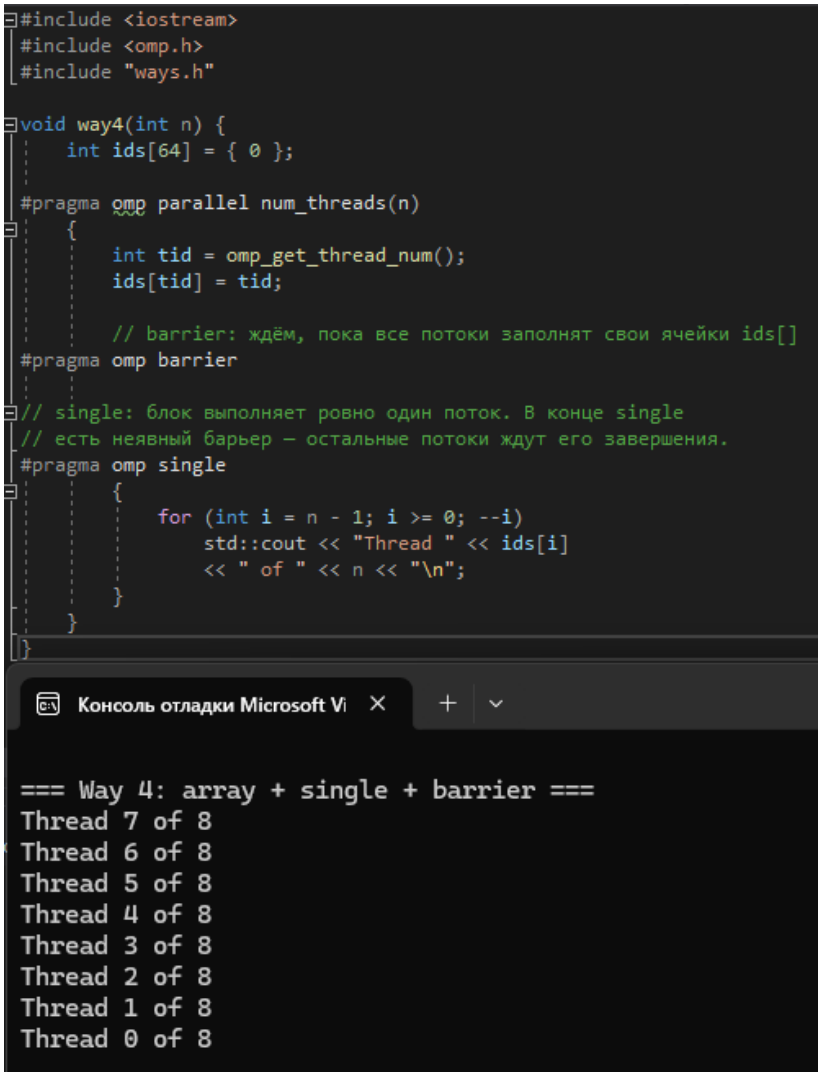


Рисунок 8 — Код способа 4: массив + single + barrier (way4.cpp)

5 способ – single. Похож на способ 4: потоки заполняют массив ids[tid] = tid, синхронизируются барьером, а затем печатает массив **только поток с** tid == 0. Различие — в директиве, которая выбирает печатающий поток.

Механизм синхронизации. #pragma omp barrier и #pragma omp master. Директива master гарантирует, что блок выполнит именно поток 0 (в отличие от single, где поток выбирается произвольно). Важное отличие master от single: в конце master нет неявного барьера, тогда как single его создаёт. В данной задаче это не критично, потому что после master параллельная область завершается сама по себе; но при наличии кода после блока разница была бы существенной. Работа программы представлена на рисунке 9.

#include <iostream>

```
#include <omp.h>
#include "ways.h"
void way5(int n) {
   int ids[64] = { 0 };
#pragma omp parallel num_threads(n)
   {
      int tid = omp_get_thread_num();
      ids[tid] = tid;
#pragma omp barrier
#pragma omp master
      {
         for (int i = n - 1; i >= 0; --i)
            std::cout << "Thread " << ids[i]
            << " of " << n << "\n";
      }
   }
}
```

```
#include <iostream>
#include <omp.h>
#include "ways.h"

void way5(int n) {
    int ids[64] = { 0 };

#pragma omp parallel num_threads(n)
    {
        int tid = omp_get_thread_num();
        ids[tid] = tid;

#pragma omp barrier   // синхронизация: все ids[] заполнены

        // master: блок выполняет только поток 0.
        // В отличие от single, в конце master НЕТ неявного барьера.
#pragma omp master
        {
            for (int i = n - 1; i >= 0; --i)
                std::cout << "Thread " << ids[i]
                << " of " << n << "\n";
        }
    }
}
```

```
Thread 0 of 8

=== Way 5: array + master + barrier ===
Thread 7 of 8
Thread 6 of 8
Thread 5 of 8
Thread 4 of 8
Thread 3 of 8
Thread 2 of 8
Thread 1 of 8
Thread 0 of 8

=== Way 6: atomic capture ===
Thread 7 of 8
Thread 6 of 8
Thread 5 of 8
Thread 4 of 8
Thread 3 of 8
Thread 2 of 8
Thread 1 of 8
Thread 0 of 8
```

Рисунок 9 — Код способа 5: массив + master + barrier (way5.cpp)

6 способ - Логика та же, что и в способе 2: разделяемая переменная counter инициализируется значением n, каждый поток уменьшает её на единицу. Отличие — в том, что внутри критической секции выполняется сразу два действия: декремент counter и сохранение результата в локальную переменную my_id. Затем во второй критической секции поток печатает my_id.

Механизм синхронизации. Две директивы #pragma omp critical. Первая защищает пару «декремент + чтение» — это эквивалент atomic capture из OpenMP 3.1, но реализованный через критическую секцию. Вторая защищает печать, чтобы строки разных потоков не перемешивались.

Такая конструкция совместима с OpenMP 2.0, который поддерживается в Visual Studio 2019 без дополнительных ключей. Дело в том, что конструкция #pragma omp atomic capture появилась только в OpenMP 3.1, и в VS

2019 в режиме /openmp она не компилируется (ошибка C3005). Поэтому декремент и чтение объединены в critical — по результату это эквивалентно, но собирается без проблем. Работа программы представлена на рисунке 10.

```
#include <iostream>
#include <omp.h>
#include "ways.h"
void way6(int n) {
   int counter = n;
#pragma omp parallel num_threads(n)
   {
      int my_id;
#pragma omp critical
      {
         --counter;
         my_id = counter;
      }
#pragma omp critical
      std::cout << "Thread " << my_id
         << " of " << n << "\n";
   }
}
```

```
#include <iostream>
#include <omp.h>
#include "ways.h"

void way6(int n) {
    int counter = n;   // разделяемая переменная

#pragma omp parallel num_threads(n)
    {
        int my_id;

        // critical: атомарно уменьшаем счётчик и сразу запоминаем резул
        // Это надёжно работает в OpenMP 2.0 (VS 2019), в отличие от ato
#pragma omp critical
        {
            --counter;
            my_id = counter;
        }

        // Печать — тоже в critical, чтобы строки не перемешались.
        // (Можно объединить с блоком выше, но так нагляднее.)
#pragma omp critical
        std::cout << "Thread " << my_id
            << " of " << n << "\n";
    }
}
```

```
=== Way 5: array + master + barrier ===
Thread 7 of 8
Thread 6 of 8
Thread 5 of 8
Thread 4 of 8
Thread 3 of 8
Thread 2 of 8
Thread 1 of 8
Thread 0 of 8

=== Way 6: atomic capture ===
Thread 7 of 8
Thread 6 of 8
Thread 5 of 8
Thread 4 of 8
Thread 3 of 8
Thread 2 of 8
Thread 1 of 8
Thread 0 of 8

C:\labs\zadanie3\Debug\zadanie3.exe (процесс
```

Рисунок 9 — Код способа 5: массив + master + barrier (way5.cpp)